

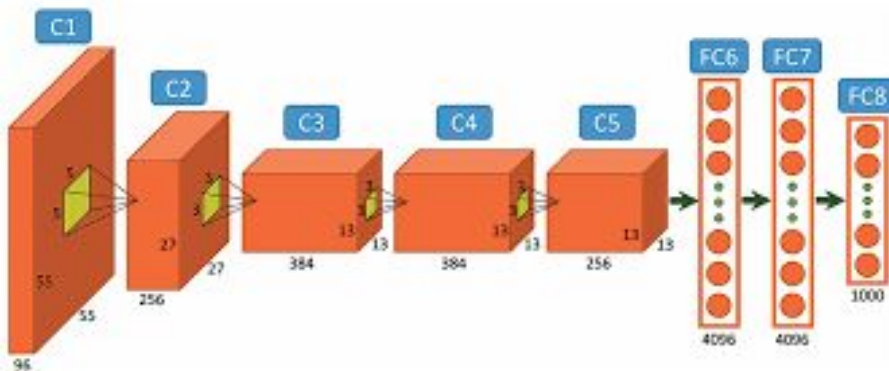
Summer Institute in Computational Social Sciences

Cap. 2. Transformers



Un poquito de historia...

- Hacia 2010-2011, la “vedette” de Deep Learning era Computer Vision: AlexNet, DanNet
- NLP siempre iba un poco atrás: pocas aplicaciones de Deep Learning
- Primer hito: word embeddings



Un poquito de historia...

- A partir de allí NLP y Redes Neuronales se empezaron a llevar bien.
- Redes Neurnales Recursivas (RNN) y Long-Short Term Memory (LSTM)
- Las RNN aprenden una función autorregresiva

$$Y_t = f(x_t, y_{t-1})$$

Cambio en el “paradigma” de NLP

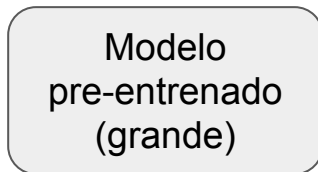
- Hasta 2013-2014 => un modelo para cada tarea

Transfer learning

- Entrenar modelo de lenguaje en dataset grande
- Adaptar el LM a nuevo dominio (por ej, IMDb)
- Entrenar para la tarea (agregando una nueva capa)

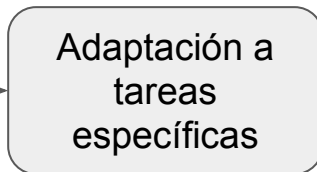
Reutilizamos la red casi entera para la tarea.

**LENTO PERO
REUTILIZABLE**

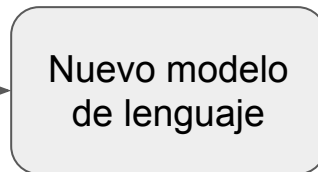


Datasets grandes,
por ejemplo
Wikipedia

MÁS RÁPIDA
Se hace para cada tarea



Directamente
sobre el dataset
de nuestro
problema



Un ejemplo

INPUT

Je suis étudiant

Traducción
Problema Sequence to Sequence

OUTPUT

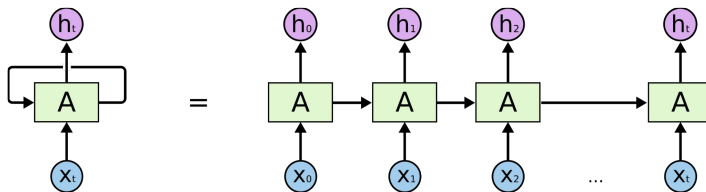
I am a student



Un ejemplo

INPUT

Je suis étudiant



OUTPUT

I am a student



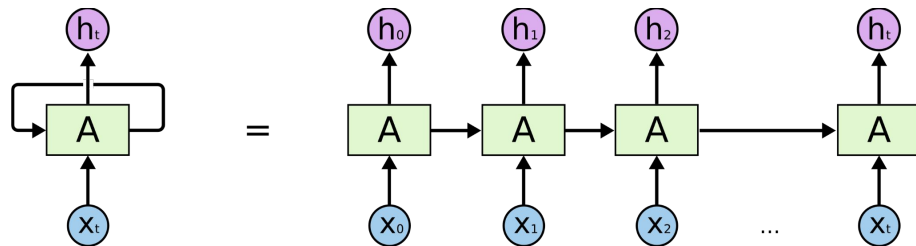
Un ejemplo

- El modelo presentado hasta ahora no tiene memoria: los inputs se presentan de manera independiente y no se tiene en cuenta relación entre ellos.
- Cuando leemos texto, esto no es así. Procesamos las letras, las palabras y las oraciones teniendo en cuenta la información que leímos previamente.
- Las **Recurrent Neural Networks** imitan esta lógica → primeros modelos de trabajo con texto.



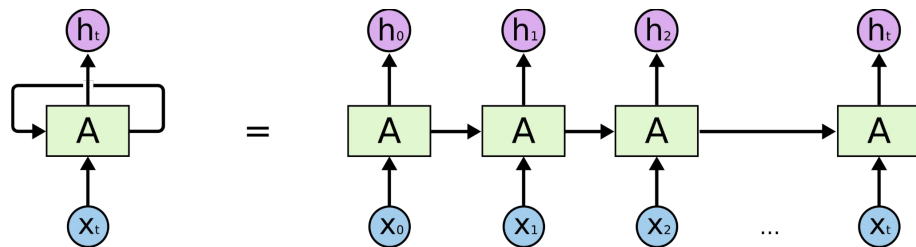
Un ejemplo - RNN

- Aprendizaje secuencial, tiene loop interno y va aprendiendo sobre lo que ya vio.
 - Sigue un loop interno. En cada iteración considera el estado actual del input y lo introduce (hidden state) para obtener output.



Un ejemplo - RNN

- Limitaciones
 - Es secuencial, loop que pasa de una etapa a la otra.
 - No hay una *paralelización* del aprendizaje, o sea, no capturan relaciones globales en un texto.



Un ejemplo



Transformers

- Modelo paralelizable → puede procesar varias partes de una secuencia al mismo tiempo, lo que acelera considerablemente el entrenamiento y la inferencia.
- Capta las dependencias a largo plazo en el texto, lo que permite comprender mejor el contexto general y generar textos más coherentes.
- Utiliza mecanismos de **self-attention** [Vaswani et al \(2017\)](#).

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

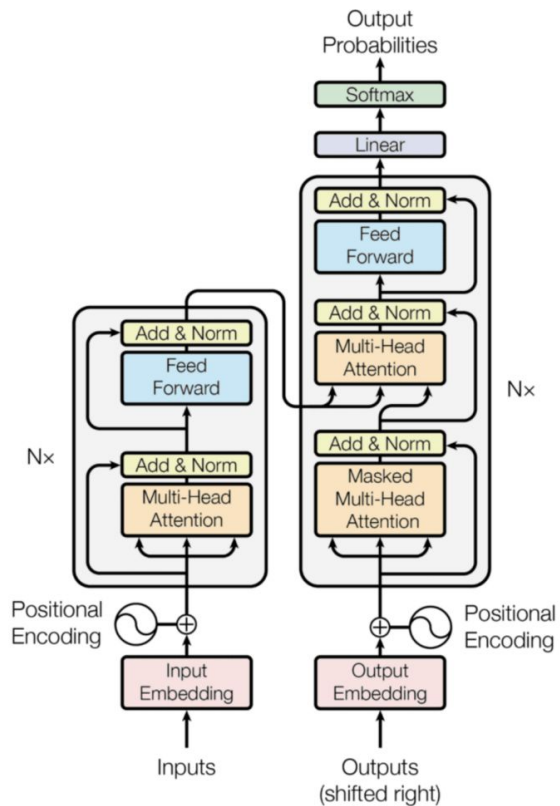
Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

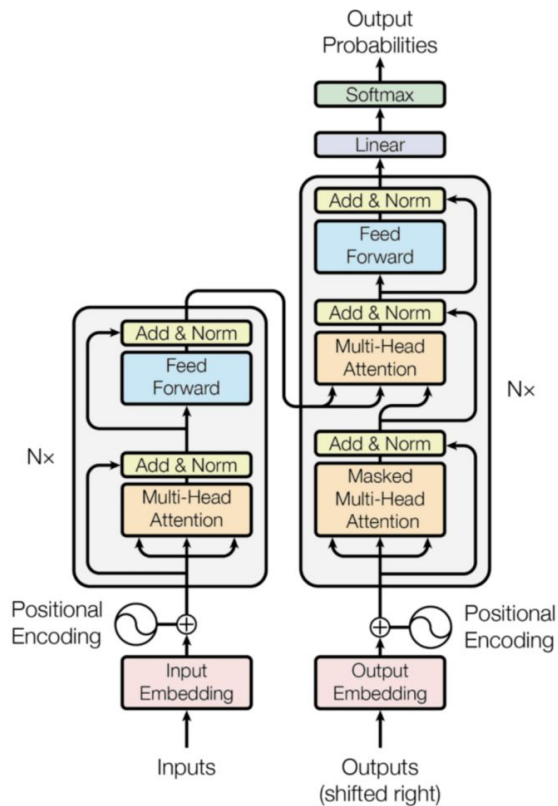


Transformers

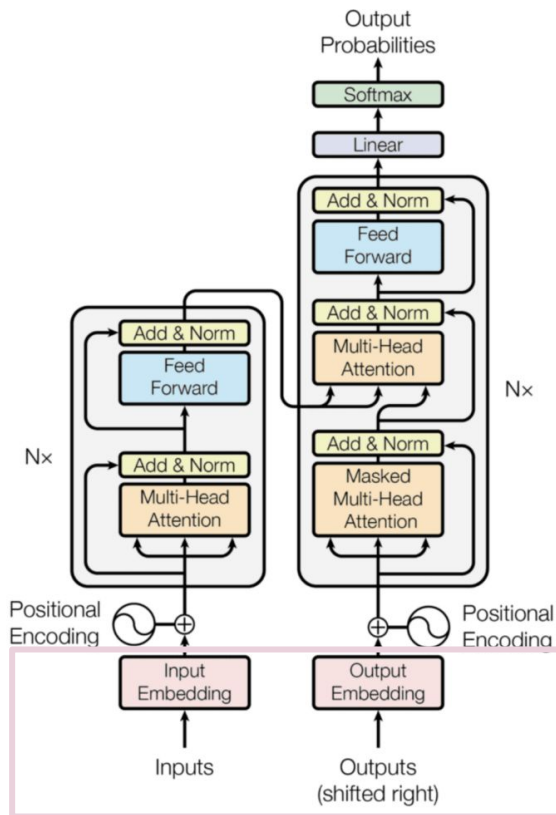


Transformers

Tres mecanismos importantes



Transformers

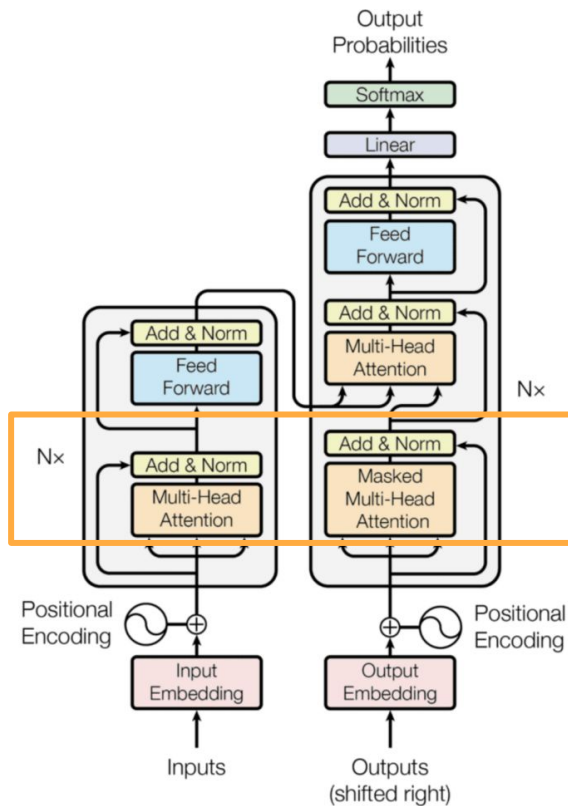


Tres mecanismos importantes

- Input/Output Embeddings



Transformers

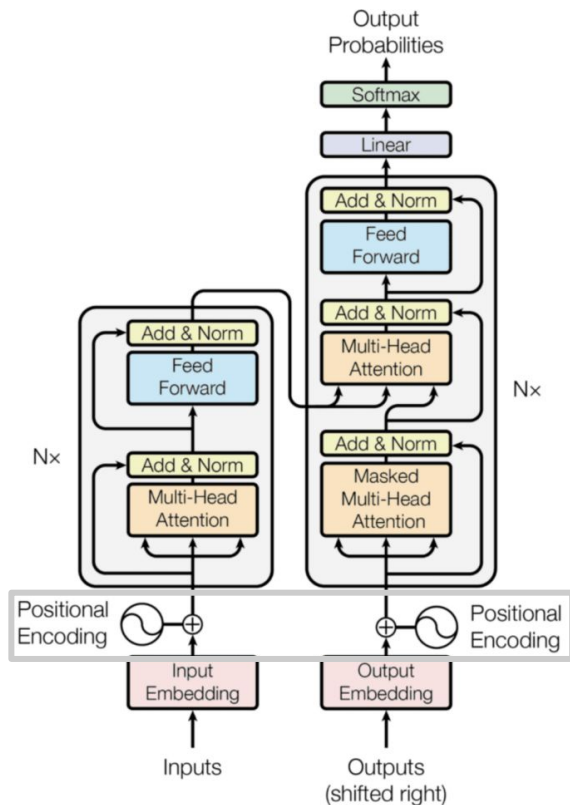


Tres mecanismos importantes

- Input/Output Embeddings
- Multi-head Attention



Transformers

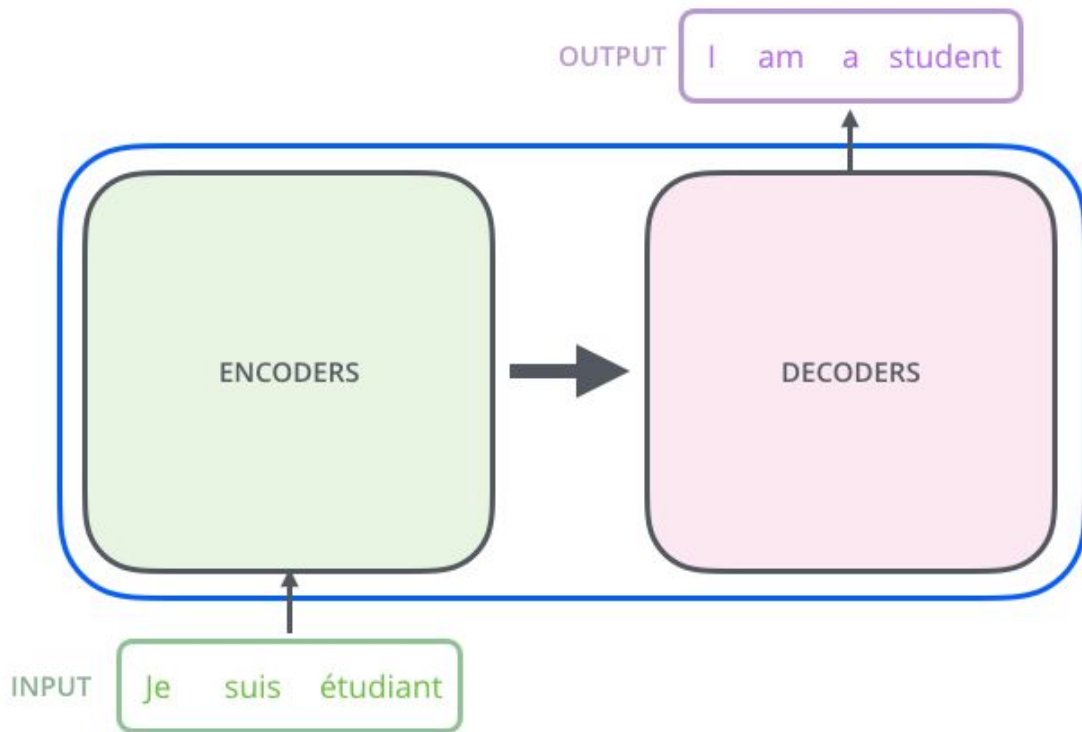


Tres mecanismos importantes

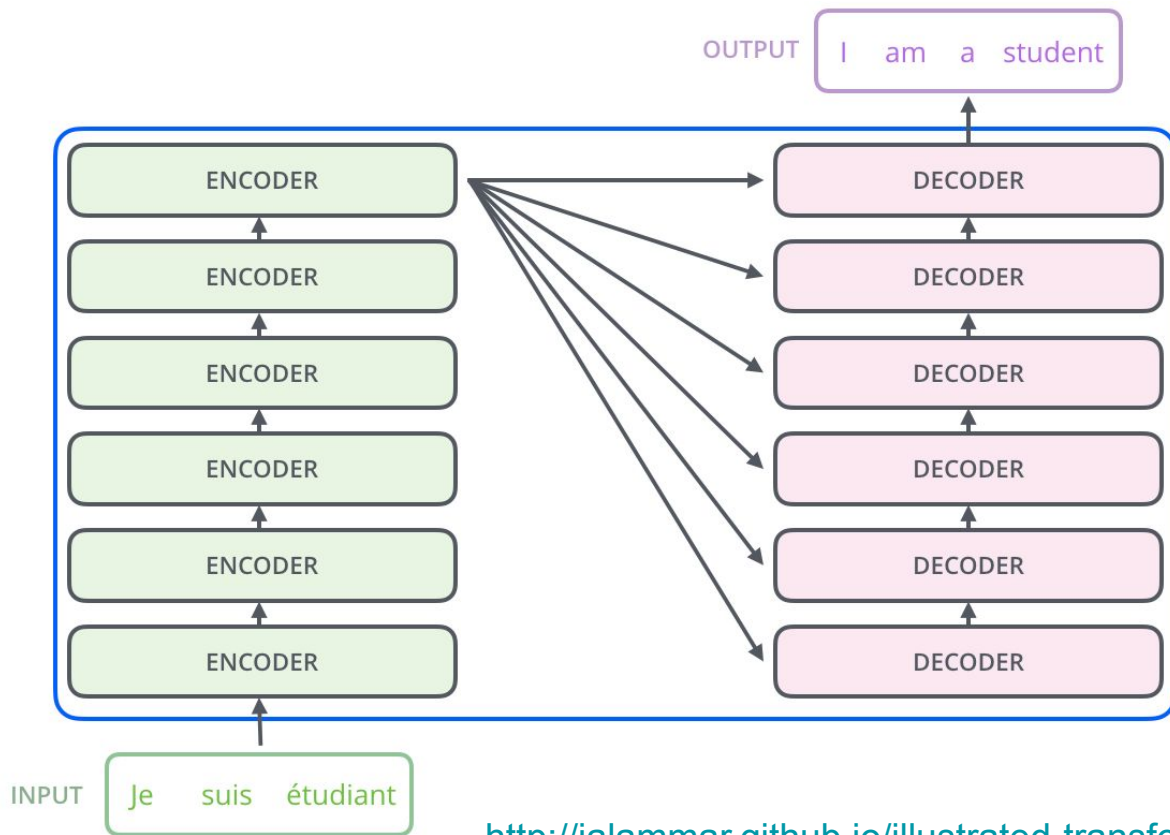
- Input/Output Embeddings
- Multi-head Attention
- Positional encoding



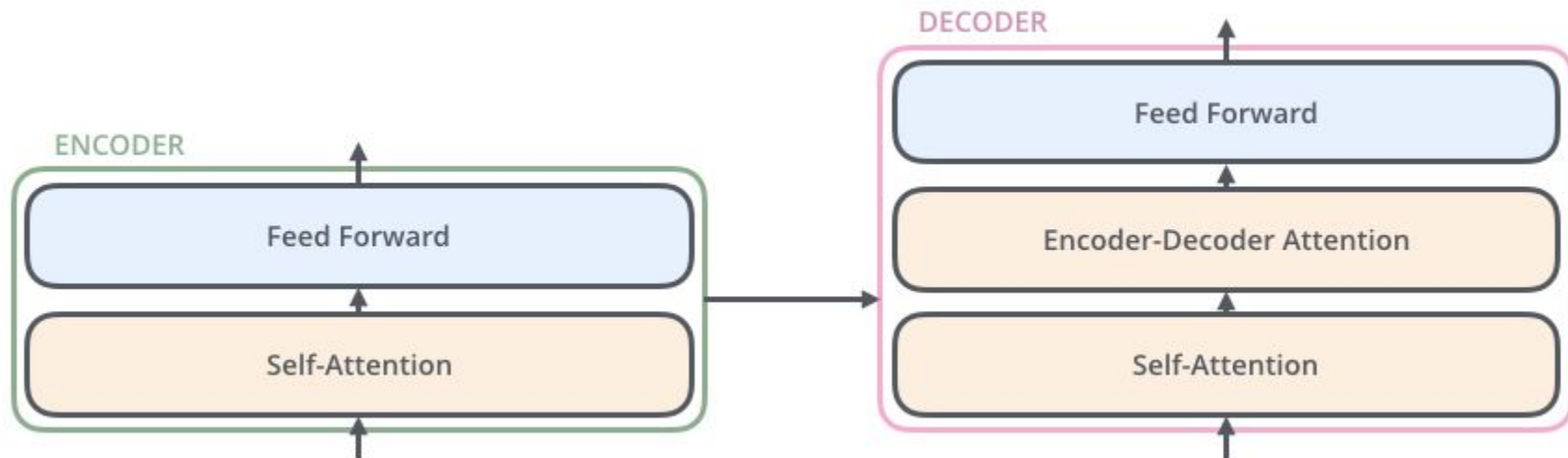
Abriendo la caja (un poquito)



Abriendo la caja



Abriendo la caja

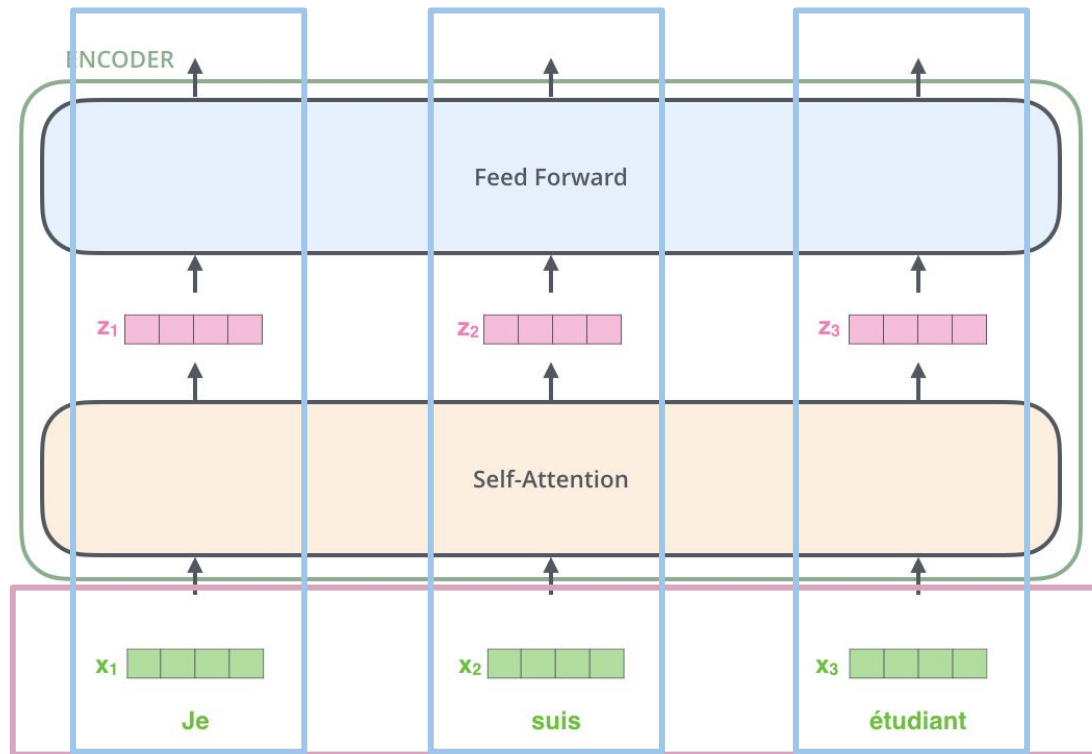


Abriendo la caja

Cada palabra “fluye” de forma paralela a través del encoder.

¿Cómo se recuperan las dependencias de palabras? =>
Self-Attention mechanism

Word Embedding
(d = hiperparámetro)
Se entrena con el modelo



Self-attention

“El perro no jugó con el niño porque él tenía pulgas”

- ¿A quién remite el término “él”? ¿Al perro o al niño?
- Para nosotros es evidente, pero para un modelo no.
- Cuando el modelo procesa la palabra "él", la atención propia le permite asociarla con “perro”.
- A medida que el modelo procesa cada palabra (cada posición en la secuencia de entrada), *self-attention* le permite buscar otras posiciones en la secuencia de entrada en busca de pistas que puedan ayudar a codificar mejor esta palabra.



Self-attention

- Cada input se asocia a tres vectores:
 - Query (Q), Key (K) y Value (V).
 - Los vectores surgen de multiplicar cada embedding de cada palabra por una matriz de pesos (W_Q , W_K y W_V) que se aprenden durante el entrenamiento.
- Se calculan las puntuaciones de similitud entre los vectores de Q y K.
 - Indican cuánta atención debe prestarse a cada elemento de la secuencia al procesar el elemento actual.
- Suma ponderada: Las puntuaciones de atención se utilizan para calcular una suma ponderada de los vectores. Esta suma ponderada representa el contexto o la información de toda la secuencia de entrada relevante para el elemento actual.



Self-attention

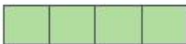
Son producto del entrenamiento...

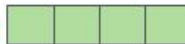
Input

Thinking

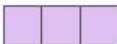
Machines

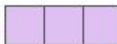
Embedding

X_1 

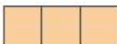
X_2 

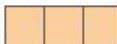
Queries

q_1 

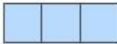
q_2 

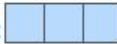
Keys

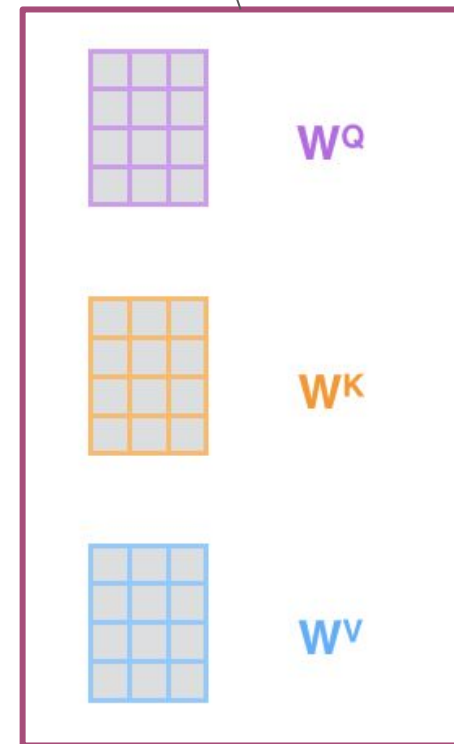
k_1 

k_2 

Values

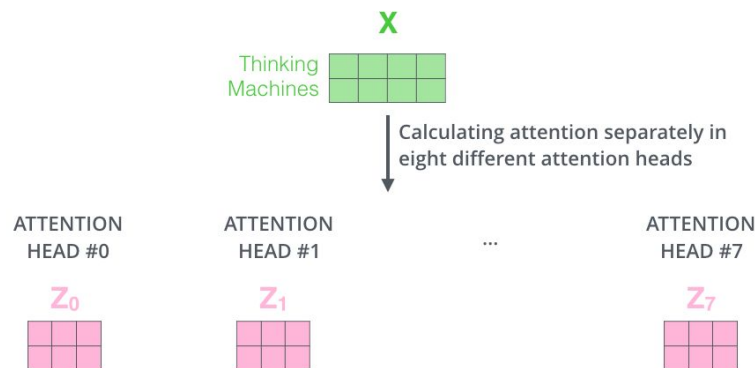
v_1 

v_2 



Multihead self-attention

- Atención multicabezal: La autoatención se aplica normalmente en paralelo varias veces con diferentes conjuntos de vectores Q, K y V aprendidos, creando múltiples "cabezas de atención". En el paper se aplica 8 veces por lo cual tenemos $3 \times 8 = 24$ matrices para aprender por cada capa en el entrenamiento.
- Esto permite al modelo centrarse en diferentes aspectos de los datos de entrada y capturar varios tipos de relaciones.

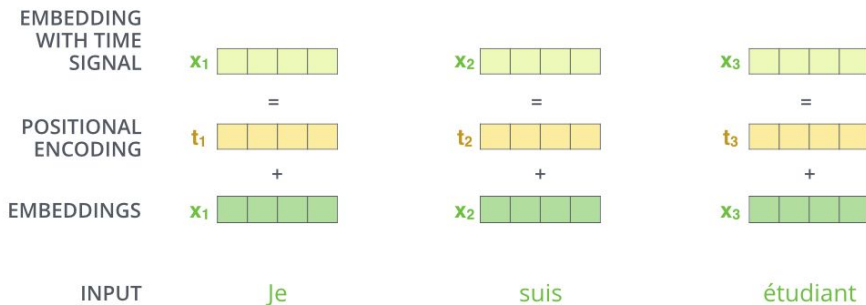
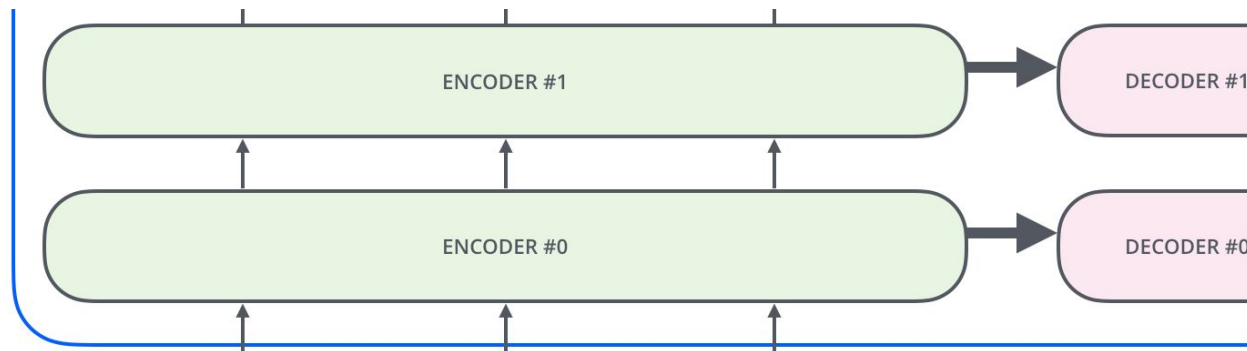


Positional encoding

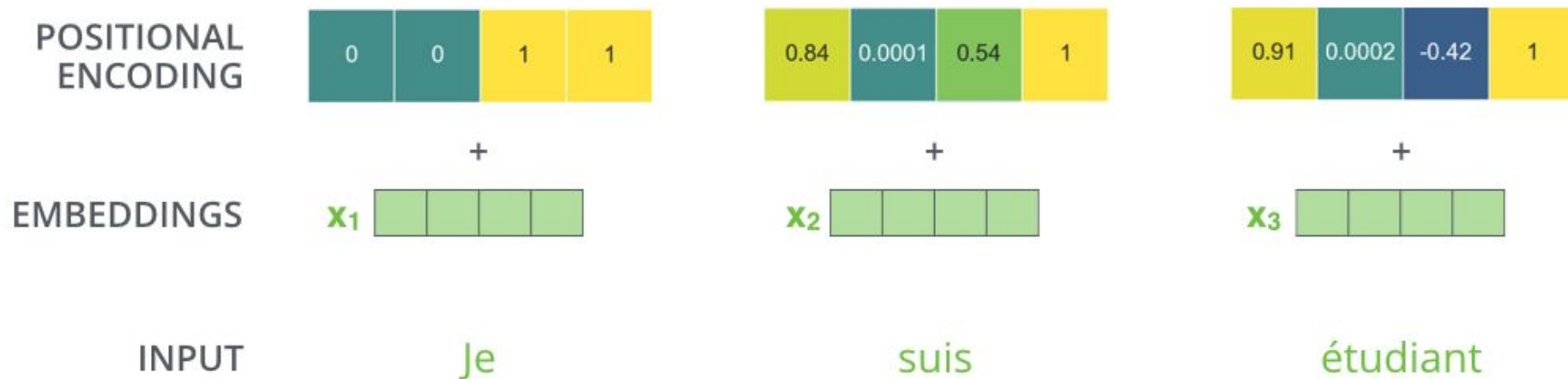
Nos falta algo:
necesitamos poder
identificar el orden o la
posición de cada palabra
en la secuencia de input.

Para esto, el modelo
agrega un vector a cada
uno de los embeddings
de input.

En el paper original, el
positional embedding de
cada posición (1, 2, ...
hasta un M fijo) se
computa con un cálculo
fijo

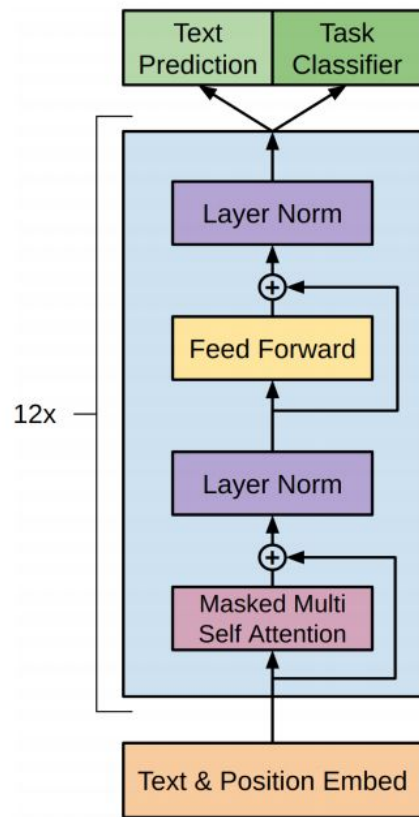


Positional encoding



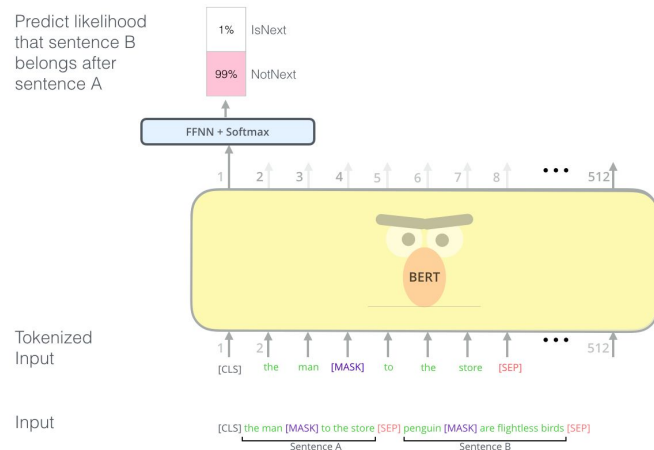
Modelos basados en Transformers: GPT

- Radford et al (2018) entrenaron un modelo de lenguaje basado en 12 capas de Transformers
- Sacamos la última capa y la reemplazamos por una capa para clasificación
- Entrenamos el modelo completo con un learning rate bajo
- Pequeño detalle: como el modelo de lenguaje sólo puede mirar para atrás, se usan máscaras en la atención
- State-of-the-art en GLUE (benchmark de varias tareas de NLP)



Modelos basados en Transformers: BERT

- Misma idea que GPT, pero en vez de entrenar un modelo de lenguaje, usan otras dos tareas:
 - Predecir un token enmascarado (Masked Language Model)
 - Ver si una oración es la que le sigue a otra (Next-Sentence prediction)
- Entrenado sobre Wikipedia o Common Crawl
- Acá no usamos ninguna máscara ya que todos los tokens pueden ver a los demás
- Sobrepasó a GPT en el GLUE



La evolución de los transformers



Vamos al notebook...



Parte Hands-on



Consignas orientadoras

1. Cargar el dataset de comentarios
2. Preprocesarlo
3. Utilizar pysentimiento para clasificar los comentarios según polaridad ('task=sent')
4. Diseñar y ejecutar una estrategia de validación de los resultados
5. Joinear con los resultados del modelado de tópicos y efectuar un análisis exploratorio. ¿Hay temas que son polarizantes? ¿Cuáles son? ¿Cómo evolucionan a lo largo del tiempo?

